can be on the left side of an assignment expression—named objects, objects allocated on the stack or heap, or members of other objects—things with a defined storage location. The term *rvalue* also comes from C and refers to things that can occur only on the right side of an assignment expression—literals and temporaries, for example. Lvalue references can only be bound to lvalues, not rvalues. You can't write

```
int& i=42;      <-- Won't compile
```

for example, because `42` is an rvalue. OK, that's not quite true; you've always been able to bind an rvalue to a `const` lvalue reference:

```
int const& i=42;
```

But this is a deliberate exception on the part of the standard, introduced before we had rvalue references in order to allow you to pass temporaries to functions taking references. This allows implicit conversions, so you can write things like this:

```
void print(std::string const& s);
print("hello");          <-- Create temporary std::string object
```

Anyway, the C++11 Standard introduces *rvalue references*, which bind *only* to rvalues, not to lvalues, and are declared with two ampersands rather than one:

```
int&& i=42;
int j=42;
int&& k=j;          <-- Won't compile
```

You can thus use function overloading to determine whether function parameters are lvalues or rvalues by having one overload take an lvalue reference and another take an rvalue reference. This is the cornerstone of *move semantics*.

A.1.1. Move semantics

Rvalues are typically temporary and so can be freely modified; if you know that your function parameter is an rvalue, you can use it as temporary storage,